UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

FACULTAD DE INGENIERÍA ELÉCTRICA, ELECTRÓNICA, INFORMÁTICA Y MECÁNICA

ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE SISTEMAS



Proyecto: Avance N.° 05

Algoritmos Avanzados

Docente: Héctor Eduardo Ugarte Rojas

Integrantes:

- Caballero Zegarra, Steffano Ricardo
- Llactahuamani Ayala, Adriel Alfredo
- Morocco Amanca, Albert Erik
- Quispe Castilla, Diego Alonso

Cusco - Perú 2026

Resumen

Este informe documenta la entrega e integración del **Avance 05** para el proyecto de ubicación óptima de Sistemas de Detección de Intrusos (IDS) modelado como Cobertura de Vértices (*Vertex Cover*). Se presenta la implementación funcional y la evaluación del segundo algoritmo del proyecto: la 2-aproximación por Emparejamiento Maximal (*Maximal Matching*), así como su integración con la heurística Greedy por Grado y el algoritmo exacto por Fuerza Bruta. El sistema cuenta con una arquitectura modular en Python (`src/`), un conjunto de 27 pruebas unitarias e integrales automatizadas con `pytest` (cobertura del 100 % en casos básicos, límite y adversarios) y una interfaz de línea de comandos unificada (`demo.py`) que maneja errores y entradas inválidas de forma interactiva. Se incluye el análisis empírico sobre instancias pequeñas y adicionales de escala ($n = 6$ a $n = 18$), demostrando la validez de los resultados mediante el módulo unificado de verificación (`verify.py`) y justificando teóricamente las diferencias observadas en la calidad de las soluciones devueltas por ambas alternativas.

Índice

1 Introducción y Contexto del Proyecto **2**

2 Arquitectura del Sistema e Integración Modular **2**
2.1 Estructura de Carpetas del Repositorio 2
2.2 Módulos Reutilizados 2

3 Implementación Funcional del Segundo Algoritmo: Matching Maximal **3**
3.1 Lógica y Pseudocódigo 3
3.2 Garantía Matemática de la Cota 2× 3
3.3 Garantía de Determinismo y Reproducibilidad 3

4 Interfaz Común (CLI) y Manejo de Entradas Inválidas **4**
4.1 Sintaxis de Ejecución 4
4.2 Manejo Robusto de Excepciones 4

5 Demostración Práctica e Interpretación de Resultados **4**
5.1 Instancia Pequeña Procesada (`example_small.json`) 4
5.2 Análisis Crítico de Diferencias entre Algoritmos 5
5.3 Pruebas Automatizadas con `pytest` 5
5.4 Resultados Experimentales de Escalabilidad (`compare.py`) 5

6 Matriz de Cumplimiento de Premisas (Avance 05) **6**

7 Conclusiones **6**

1. Introducción y Contexto del Proyecto

En el diseño de arquitecturas de seguridad para redes de comunicaciones, la inspección de tráfico mediante Sistemas de Detección de Intrusos (IDS) o cortafuegos resulta indispensable. Sin embargo, instalar dispositivos IDS en la totalidad de los routers de la topología genera costos elevados e innecesarios de licencias y mantenimiento. Basta con que cada enlace físico tenga al menos uno de sus extremos monitoreado para garantizar visibilidad sobre todo paquete que circula por la red.

Este escenario se modela de manera natural como el problema de **Cobertura de Vértices** (*Vertex Cover*): dado un grafo no dirigido $G = (V, E)$, encontrar el subconjunto de cardinalidad mínima $C \subseteq V$ tal que $\forall (u, v) \in E$, se cumple $u \in C \vee v \in C$.

Al ser un problema conocido NP-difícil, no existe una solución polinomial exacta en el caso general (asumiendo $P \neq NP$). Por tanto, la línea de trabajo asignada requiere desarrollar, integrar y comparar múltiples estrategias de solución:

1. **Primera alternativa (Greedy por Grado):** Heurística voraz orientada a seleccionar en cada paso el vértice con mayor número de aristas pendientes.
2. **Segunda alternativa (Matching Maximal):** Algoritmo de 2-aproximación con garantía matemática demostrable ($|C| \leq 2 \cdot |\text{OPT}|$).
3. **Referencia Exacta (Fuerza Bruta):** Algoritmo exponencial de validación para instancias pequeñas ($|V| \leq 20$).

2. Arquitectura del Sistema e Integración Modular

El proyecto se ha construido bajo una arquitectura de software modular y desacoplada en Python. Los componentes desarrollados se reutilizan transversalmente para la lectura de datos, generación de instancias, ejecución algorítmica y verificación de resultados.

2.1. Estructura de Carpetas del Repositorio

A continuación se detalla la organización del repositorio en el sistema de archivos:

```
vertex-cover-ids/
 src/
  graph.py # Estructura de grafo (lista de adyacencia) + I/O + generadores
  solver.py # VertexCoverSolver: Greedy, Matching Maximal y Exacto
  verify.py # Mdulo de verificacin de coberturas e indicadores
 tests/
  test_graph.py # Pruebas de la estructura de datos
  test_solver.py # Suite de 27 pruebas bsicas, lmite y adversarias
 data/
  instances/ # Archivos de prueba JSON (ej. example_small.json)
 experiments/
  compare.py # Medicin de tiempo, tamao y ratio de aproximacin
  results/ # Salidas cuantitativas (comparison.csv)
 demo.py # Interfaz de comandos unificada (CLI)
 requirements.txt # Dependencias del proyecto (pytest)
```

2.2. Módulos Reutilizados

- **Estructura de Datos (`src/graph.py`):** Utiliza una lista de adyacencia mediante un diccionario de conjuntos (`dict[Hashable, set]`), optimizando las consultas de grado a $O(1)$ y la

iteración de vecinos a $O(\deg(v))$. Reutilizado tanto por los algoritmos como por el generador de grafos sintéticos (modelos de Erdős-Rényi y Barabási-Albert).

- **Verificación Unificada (`src/verify.py`):** Proporciona la función `is_vertex_cover(graph, cover)`, la cual inspecciona la totalidad de las aristas del grafo para confirmar si la cobertura es válida. Ninguna prueba ni ejecución acepta una solución confiando únicamente en su tamaño.
- **Gestión de Archivos e I/O (`src/graph.py`):** Carga y exportación estandarizada de instancias en formato JSON.

3. Implementación Funcional del Segundo Algoritmo: Matching Maximal

El segundo algoritmo corresponde a la 2-aproximación por Emparejamiento Maximal (`maximal_matching_ve` en `src/solver.py`).

3.1. Lógica y Pseudocódigo

El algoritmo selecciona una arista arbitraria (u, v) de la red, añade ambos extremos u y v al conjunto cobertura C, y elimina del conjunto de aristas pendientes todas aquellas que sean incidentes a u o a v. El proceso se repite iterativamente hasta agotar las aristas.

Pseudocódigo - Maximal Matching (src/solver.py)

```
def maximal_matching_vertex_cover(self) -> set:
    remaining_edges = self._sorted_edges() # Orden cannico reproducible
    cover = set()

    while remaining_edges:
        u, v = remaining_edges[0]
        cover.add(u)
        cover.add(v)
        remaining_edges = [
            (a, b) for (a, b) in remaining_edges
            if a not in (u, v) and b not in (u, v)
        ]
    return cover
```

3.2. Garantía Matemática de la Cota $2\times$

Sea $M = \{(u_1, v_1), (u_2, v_2), \ldots, (u_k, v_k)\}$ el conjunto de k aristas seleccionadas por el algoritmo. Por construcción, ninguna pareja de aristas en M comparte vértices (forman un emparejamiento o *matching*). Por lo tanto, cualquier cobertura de vértices válida (incluyendo la solución óptima OPT) debe incluir al menos un vértice distinto por cada arista de M, de donde $|\text{OPT}| \geq k$.

Como el algoritmo agrega ambos extremos de cada una de las k aristas, el tamaño devuelto es $|C_{\text{match}}| = 2k$. Combinando ambas expresiones se obtiene la garantía formal:

$$|C_{\text{match}}| = 2k \leq 2 \cdot |\text{OPT}| \tag{1}$$

3.3. Garantía de Determinismo y Reproducibilidad

Durante el desarrollo se identificó que la iteración directa sobre conjuntos nativos (`set`) en Python genera órdenes de iteración variables entre ejecuciones distintas debido a la aleatorización del hash (`PYTHONHASHSEED`). Para garantizar resultados 100 % reproducibles, el segundo

algoritmo resuelve la selección .ªrbitraria"sobre una lista de aristas previamente ordenada de forma canónica (`_sorted_edges()`).

4. Interfaz Común (CLI) y Manejo de Entradas Inválidas

La interacción y demostración del sistema se centraliza en el archivo `demo.py`, el cual actúa como interfaz de línea de comandos unificada.

4.1. Sintaxis de Ejecución

La interfaz permite ejecutar cualquier algoritmo de forma individual o compararlos en paralelo utilizando la opción `-algorithm all`:

```
# Prototipo sobre una instancia pequea conocida
python demo.py --instance data/instances/example_small.json --algorithm all

# Prototipo sobre una instancia generada al vuelo
python demo.py --random 12 0.3 --seed 42 --algorithm all
```

4.2. Manejo Robusto de Excepciones

`demo.py` implementa validación de parámetros y captura de errores para prevenir fallos en tiempo de ejecución:

- **Archivos no encontrados:** Atrapa errores de ruta y notifica si el archivo JSON no existe.
- **Instancias sobredimensionadas en Fuerza Bruta:** Cuando el número de vértices supera $n > 20$, el método `exact_vertex_cover_bruteforce` lanza un `ValueError` controlado, evitando bloqueos indefinidos por complejidad exponencial.
- **Parámetros de generación aleatoria:** Valida que la probabilidad de arista p se encuentre strictly en el intervalo $[0, 1]$ y que el número de vértices n sea un entero positivo.

5. Demostración Práctica e Interpretación de Resultados

5.1. Instancia Pequeña Procesada (`example_small.json`)

A continuación se presenta la traza real obtenida al ejecutar la demostración en la terminal sobre la mini-red de 6 routers ($|V| = 6, |E| = 6$):

```
Cargado: instancia 'data/instances/example_small.json'
  |V| = 6 |E| = 6

[Greedy por grado]
  Cobertura: ['A', 'B', 'E']
  Tamano: 3
  Valida: SI
  Tiempo: 0.048 ms

[Matching maximal (2-aprox)]
  Cobertura: ['A', 'B', 'D', 'E']
  Tamano: 4
  Valida: SI
  Tiempo: 0.029 ms

[Exacto (fuerza bruta)]
```

```
  Cobertura: ['A', 'B', 'E']
  Tamano: 3
  Valida: SI
  Tiempo: 0.059 ms

Ratios de aproximacion frente al optimo:
  Greedy por grado: 1.000x
  Matching maximal (2-aprox): 1.333x
```

5.2. Análisis Crítico de Diferencias entre Algoritmos

Como se observa en la salida de la instancia pequeña:

1. **Equivalencia de Validez:** Ambos algoritmos devuelven soluciones válidas (`Valida: SI`), verificado por el módulo `verify.is_vertex_cover`.
2. **Diferencia de Tamaños:** El Greedy por grado obtiene una cobertura de tamaño 3 (1,000×), igualando la solución óptima exacta. El Matching Maximal obtiene una cobertura de tamaño 4 (1,333×).
3. **Justificación Teórica:** La diferencia de cardinalidad **no es un error de implementación**, sino una **propiedad teórica esperada**. El Greedy selecciona de forma voraz el nodo de mayor grado actual, mientras que el Matching Maximal añade ambos extremos de una arista independiente para garantizar la cota matemática 2× en el peor caso. En grafos con nodos concentradores (*hubs*), tomar ambos extremos añade vértices redundantes, pero asegura que el algoritmo nunca supere $2 \cdot |\text{OPT}|$.

5.3. Pruebas Automatizadas con `pytest`

El repositorio cuenta con 27 pruebas automatizadas que validan la correcta integración de todos los componentes. Los resultados en la terminal confirman un 100 % de éxito:

```
tests/test_graph.py ....... [ 25%]
tests/test_solver.py ::TestCasosBasicos PASSED [ 33%]
tests/test_solver.py ::TestCasosLimite PASSED [ 55%]
tests/test_solver.py ::TestCasosAdversarios PASSED [100%]
====================== 27 passed in 0.11s ======================
```

5.4. Resultados Experimentales de Escalabilidad (`compare.py`)

Se ejecutó el script de comparación sobre grafos aleatorios $G(n, p = 0{,}3)$ variando n desde 6 hasta 18. La Tabla 1 resume los resultados.

Tabla 1: Resultados del experimento comparativo de escalabilidad.

n	$\|E\|$	**Óptimo**	**Greedy (Ratio)**	**Matching (Ratio)**	t_{exacto} **(ms)**
6	3	2	2 (1.00x)	4 (2.00x)	0.02 ms
8	10	4	4 (1.00x)	6 (1.50x)	0.07 ms
10	13	5	6 (1.20x)	10 (2.00x)	0.28 ms
12	19	5	5 (1.00x)	10 (2.00x)	0.78 ms
14	35	9	9 (1.00x)	12 (1.33x)	8.76 ms
16	33	9	9 (1.00x)	14 (1.56x)	25.87 ms
18	52	12	12 (1.00x)	16 (1.33x)	159.61 ms

6. Matriz de Cumplimiento de Premisas (Avance 05)

Para facilitar la evaluación por parte del docente, la Tabla 2 mapea cada una de las premisas requeridas por la guía de la actividad con su correspondiente implementación en el repositorio.

Tabla 2: Matriz de trazabilidad y cumplimiento de los requisitos del Avance 05.

Premisa / Requisito del Avance 05	**Estado**	**Ubicación en Código / Evidencia**
1. Implementación funcional del segundo algoritmo	Cumplido	Método `maximal_matching_vertex_cover` en `src/solver.py`.
2. Mismas entradas, formatos y criterios de salida	Cumplido	Ambas alternativas reciben la clase `Graph` y devuelven un `set` de vértices.
3. Integración de componentes desarrollados	Cumplido	Arquitectura modular desacoplada en `src/` (`graph`, `solver`, `verify`).
4. Interfaz común para seleccionar/ejecutar	Cumplido	Script `demo.py` CLI con argumento `-algorithm all`.
5. Manejo de entradas inválidas	Cumplido	Captura de excepciones e inspección de rangos en `demo.py` y `solver.py`.
6. Reutilización de módulos (I/O, validación, medición)	Cumplido	Reuso de `Graph.from_json`, `verify.is_vertex_cover` y `experiments/compare.py`.
7. Demostración con instancia pequeña	Cumplido	Traza de ejecución registrada sobre `data/instances/example_small.json`.
8. Demostración con instancia adicional	Cumplido	Soporte para `network_backbone_18.json` o generación al vuelo vía `-random`.
9. Explicación teórica de diferencias de resultados	Cumplido	Análisis de cota 2× vs. heurística voraz sin cota en la Sección 5.2.

7. Conclusiones

El repositorio y los componentes desarrollados cumplen al 100 % con los requisitos establecidos para la presentación del Avance 05:

1. Se integró exitosamente el segundo algoritmo de solución (*Matching Maximal*) manteniendo la compatibilidad estricta en formatos de entrada y salida con la heurística Greedy y la Fuerza Bruta.

2. La suite de 27 pruebas automatizadas con `pytest` garantiza que los algoritmos funcionen de forma determinista y sin violar la validez de las coberturas ni las cotas teóricas.

3. La interfaz unificada `demo.py` permite realizar demostraciones interactivas en vivo, gestionando posibles errores de entrada de forma robusta.

4. Las diferencias observadas en los resultados empíricos se encuentran plenamente justificadas por el análisis teoricocomputacional de cada alternativa.